

Basic Statistics

```
In [43]: import pandas as pd
```

```
In [44]: df=pd.read_csv("world-happiness-report - world-happiness-report.csv")
```

```
In [45]: df.head()
```

```
Out[45]:
```

	Country name	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	Perceptions of corruption
0	Afghanistan	2008	3.724	7.370	0.451	50.80	0.718	0.168	
1	Afghanistan	2009	4.402	7.540	0.552	51.20	0.679	0.190	
2	Afghanistan	2010	4.758	7.647	0.539	51.60	0.600	0.121	
3	Afghanistan	2011	3.832	7.620	0.521	51.92	0.496	0.162	
4	Afghanistan	2012	3.783	7.705	0.521	52.24	0.531	0.236	

```
In [46]: df.tail()
```

```
Out[46]:
```

	Country name	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	Perceptions of corruption
1944	Zimbabwe	2016	3.735	7.984	0.768	54.4	0.733	-0.095	
1945	Zimbabwe	2017	3.638	8.016	0.754	55.0	0.753	-0.098	
1946	Zimbabwe	2018	3.616	8.049	0.775	55.6	0.763	-0.068	
1947	Zimbabwe	2019	2.694	7.950	0.759	56.2	0.632	-0.064	
1948	Zimbabwe	2020	3.160	7.829	0.717	56.8	0.643	-0.009	

```
In [47]: df.shape
```

```
Out[47]: (1949, 11)
```

```
In [48]: df.columns
```

```
Out[48]: Index(['Country name', 'year', 'Life Ladder', 'Log GDP per capita',
               'Social support', 'Healthy life expectancy at birth',
               'Freedom to make life choices', 'Generosity',
               'Perceptions of corruption', 'Positive affect', 'Negative affect'],
              dtype='object')
```

```
In [49]: df.sample(8)
```

Out[49]:

	Country name	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	Perceptions of corruption
13	Albania	2009	5.485	9.262	0.833	66.20	0.525	-0.158	
686	Honduras	2007	5.097	8.500	0.819	64.78	0.676	0.230	
1727	Togo	2006	3.202	7.078	0.435	49.26	0.628	-0.007	
653	Guatemala	2009	6.452	8.891	0.834	61.76	0.643	0.197	
1301	Norway	2008	7.632	11.042	0.936	71.56	0.947	0.018	
976	Lebanon	2014	5.233	9.739	0.759	65.52	0.657	-0.012	
65	Armenia	2019	5.488	9.522	0.782	67.20	0.844	-0.172	
662	Guatemala	2018	6.627	9.042	0.841	64.80	0.910	-0.010	

In [50]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1949 entries, 0 to 1948
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Country name                          1949 non-null   object
1   year                                  1949 non-null   int64
2   Life Ladder                           1949 non-null   float64
3   Log GDP per capita                     1913 non-null   float64
4   Social support                         1936 non-null   float64
5   Healthy life expectancy at birth       1894 non-null   float64
6   Freedom to make life choices           1917 non-null   float64
7   Generosity                            1860 non-null   float64
8   Perceptions of corruption              1839 non-null   float64
9   Positive affect                        1927 non-null   float64
10  Negative affect                        1933 non-null   float64
dtypes: float64(9), int64(1), object(1)
memory usage: 167.6+ KB
```

In [51]: df.dtypes

```
Out[51]: Country name      object
year                    int64
Life Ladder             float64
Log GDP per capita       float64
Social support           float64
Healthy life expectancy at birth  float64
Freedom to make life choices  float64
Generosity               float64
Perceptions of corruption  float64
Positive affect          float64
Negative affect          float64
dtype: object
```

In [52]: df["Country name"].dtypes

Out[52]: dtype('O')

```
In [53]: df.describe()
```

Out[53]:

	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	
count	1949.000000	1949.000000	1913.000000	1936.000000	1894.000000	1917.000000	1
mean	2013.216008	5.466705	9.368453	0.812552	63.359374	0.742558	
std	4.166828	1.115711	1.154084	0.118482	7.510245	0.142093	
min	2005.000000	2.375000	6.635000	0.290000	32.300000	0.258000	
25%	2010.000000	4.640000	8.464000	0.749750	58.685000	0.647000	
50%	2013.000000	5.386000	9.460000	0.835500	65.200000	0.763000	
75%	2017.000000	6.283000	10.353000	0.905000	68.590000	0.856000	
max	2020.000000	8.019000	11.648000	0.987000	77.100000	0.985000	

```
In [54]: df
```

Out[54]:

	Country name	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	F
0	Afghanistan	2008	3.724	7.370	0.451	50.80	0.718	0.168	
1	Afghanistan	2009	4.402	7.540	0.552	51.20	0.679	0.190	
2	Afghanistan	2010	4.758	7.647	0.539	51.60	0.600	0.121	
3	Afghanistan	2011	3.832	7.620	0.521	51.92	0.496	0.162	
4	Afghanistan	2012	3.783	7.705	0.521	52.24	0.531	0.236	
...	...	...	...	...	...	...	...	...	
1944	Zimbabwe	2016	3.735	7.984	0.768	54.40	0.733	-0.095	
1945	Zimbabwe	2017	3.638	8.016	0.754	55.00	0.753	-0.098	
1946	Zimbabwe	2018	3.616	8.049	0.775	55.60	0.763	-0.068	
1947	Zimbabwe	2019	2.694	7.950	0.759	56.20	0.632	-0.064	
1948	Zimbabwe	2020	3.160	7.829	0.717	56.80	0.643	-0.009	

1949 rows × 11 columns

```
In [55]: df.columns=df.columns.str.strip().str.lower().str.replace(" ", "_")
df.columns
```

Out[55]:

Index(['country_name', 'year', 'life_ladder', 'log_gdp_per_capita', 'social_support', 'healthy_life_expectancy_at_birth', 'freedom_to_make_life_choices', 'generosity', 'perceptions_of_corruption', 'positive_affect', 'negative_affect'], dtype='object')

Column names were cleaned for better readability and error- free coding

Insight

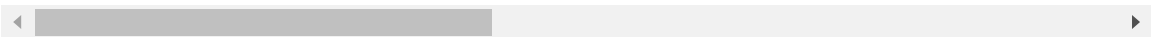
Analysed the data, cleaned column names

```
In [56]: df
```

Out[56]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1944	Zimbabwe	2016	3.735	7.984	0.768	
1945	Zimbabwe	2017	3.638	8.016	0.754	
1946	Zimbabwe	2018	3.616	8.049	0.775	
1947	Zimbabwe	2019	2.694	7.950	0.759	
1948	Zimbabwe	2020	3.160	7.829	0.717	

1949 rows × 11 columns

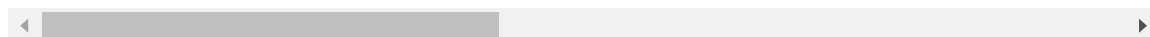


```
In [57]: df.isnull()
```

Out[57]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	False	False	False	
4	False	False	False	False	False	
...	...	...	...	...	...	...
1944	False	False	False	False	False	
1945	False	False	False	False	False	
1946	False	False	False	False	False	
1947	False	False	False	False	False	
1948	False	False	False	False	False	

1949 rows × 11 columns

In [58]: `df.isnull().sum()`

```
Out[58]: country_name      0
year      0
life_ladder      0
log_gdp_per_capita    36
social_support      13
healthy_life_expectancy_at_birth    55
freedom_to_make_life_choices      32
generosity      89
perceptions_of_corruption    110
positive_affect      22
negative_affect      16
dtype: int64
```

```
In [59]: important_cols = [
    'life_ladder',
    'log_gdp_per_capita',
    'social_support',
    'healthy_life_expectancy_at_birth',
    'freedom_to_make_life_choices'
]

for col in important_cols:
    df[col] = df[col].fillna(df[col].mean())
```

In [60]: `df.isnull().sum()`

```
Out[60]: country_name      0
         year              0
         life_ladder       0
         log_gdp_per_capita 0
         social_support    0
         healthy_life_expectancy_at_birth 0
         freedom_to_make_life_choices      0
         generosity        89
         perceptions_of_corruption         110
         positive_affect    22
         negative_affect    16
         dtype: int64
```

```
In [61]: df['generosity'] = df['generosity'].fillna(df['generosity'].mean())
         df['perceptions_of_corruption'] = df['perceptions_of_corruption'].fillna(df['perceptions_of_corruption'].mean())
```

```
In [62]: df=df.dropna()
```

```
In [63]: df.isnull().sum()
```

```
Out[63]: country_name      0
         year              0
         life_ladder       0
         log_gdp_per_capita 0
         social_support    0
         healthy_life_expectancy_at_birth 0
         freedom_to_make_life_choices      0
         generosity        0
         perceptions_of_corruption         0
         positive_affect    0
         negative_affect    0
         dtype: int64
```

```
In [64]: df.shape
```

```
Out[64]: (1925, 11)
```

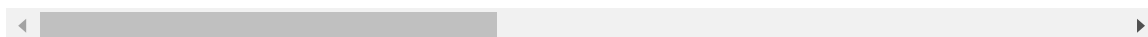
```
In [65]: df = df.drop_duplicates()
```

```
In [66]: df
```

Out[66]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_
0	Afghanistan	2008	3.724	7.370	0.451	
1	Afghanistan	2009	4.402	7.540	0.552	
2	Afghanistan	2010	4.758	7.647	0.539	
3	Afghanistan	2011	3.832	7.620	0.521	
4	Afghanistan	2012	3.783	7.705	0.521	
...	...	...	...	...	...	...
1944	Zimbabwe	2016	3.735	7.984	0.768	
1945	Zimbabwe	2017	3.638	8.016	0.754	
1946	Zimbabwe	2018	3.616	8.049	0.775	
1947	Zimbabwe	2019	2.694	7.950	0.759	
1948	Zimbabwe	2020	3.160	7.829	0.717	

1925 rows × 11 columns

**Insight**

handled missing values, removed duplicates, fixed data types

In [67]: `df.groupby('country_name')['life_ladder'].mean()`

Out[67]:

country_name	
Afghanistan	3.594667
Albania	5.019385
Algeria	5.379286
Angola	4.420250
Argentina	6.310133
...	...
Venezuela	6.019867
Vietnam	5.315923
Yemen	3.912250
Zambia	4.551714
Zimbabwe	3.882600

Name: life_ladder, Length: 164, dtype: float64

In [68]: `df.groupby('country_name').size()`

```
Out[68]: country_name
Afghanistan    12
Albania        13
Algeria         7
Angola          4
Argentina      15
..
Venezuela      15
Vietnam        13
Yemen          12
Zambia         14
Zimbabwe       15
Length: 164, dtype: int64
```

```
In [69]: df.groupby('country_name')['life_ladder'].agg(['mean', 'min', 'max'])
```

Out[69]:

	mean	min	max
country_name			
Afghanistan	3.594667	2.375	4.758
Albania	5.019385	4.511	5.867
Algeria	5.379286	4.745	6.355
Angola	4.420250	3.795	5.589
Argentina	6.310133	5.793	6.776
...	...	...	...
Venezuela	6.019867	4.041	7.478
Vietnam	5.315923	5.023	5.767
Yemen	3.912250	2.983	4.809
Zambia	4.551714	3.307	5.260
Zimbabwe	3.882600	2.694	4.955

164 rows × 3 columns

```
In [70]: df.groupby(['country_name', 'year'])['life_ladder'].mean().reset_index()
```


Out[70]:

	country_name	year	life_ladder
0	Afghanistan	2008	3.724
1	Afghanistan	2009	4.402
2	Afghanistan	2010	4.758
3	Afghanistan	2011	3.832
4	Afghanistan	2012	3.783
...	...	...	...
1920	Zimbabwe	2016	3.735
1921	Zimbabwe	2017	3.638
1922	Zimbabwe	2018	3.616
1923	Zimbabwe	2019	2.694
1924	Zimbabwe	2020	3.160

1925 rows × 3 columns

In [71]: `df.groupby('country_name')['life_ladder'].mean().reset_index()`

Out[71]:

	country_name	life_ladder
0	Afghanistan	3.594667
1	Albania	5.019385
2	Algeria	5.379286
3	Angola	4.420250
4	Argentina	6.310133
...	...	...
159	Venezuela	6.019867
160	Vietnam	5.315923
161	Yemen	3.912250
162	Zambia	4.551714
163	Zimbabwe	3.882600

164 rows × 2 columns

In [72]: `df.groupby('year')['life_ladder'].mean().reset_index()`

Out[72]:

	year	life_ladder
0	2005	6.564200
1	2006	5.202886
2	2007	5.430772
3	2008	5.417139
4	2009	5.461496
5	2010	5.499175
6	2011	5.414228
7	2012	5.443754
8	2013	5.390437
9	2014	5.375085
10	2015	5.397197
11	2016	5.399479
12	2017	5.462363
13	2018	5.507743
14	2019	5.578685
15	2020	5.859181

In [73]: `df.groupby('country_name')[['log_gdp_per_capita', 'life_ladder']].mean()`

Out[73]:

	log_gdp_per_capita	life_ladder
country_name		
Afghanistan	7.650833	3.594667
Albania	9.384385	5.019385
Algeria	9.334857	5.379286
Angola	8.990000	4.420250
Argentina	10.033800	6.310133
...	...	...
Venezuela	9.457757	6.019867
Vietnam	8.638308	5.315923
Yemen	8.313659	3.912250
Zambia	8.066857	4.551714
Zimbabwe	7.850333	3.882600

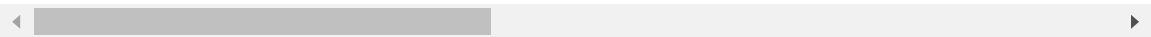
164 rows × 2 columns

In [74]: `df.sort_values(by="life_ladder")`

Out[74]:

	country_name	year	life_ladder	log_gdp_per_capita	social_support	healthy_life_
11	Afghanistan	2019	2.375	7.697	0.420	
9	Afghanistan	2017	2.662	7.697	0.491	
1667	Syria	2013	2.688	8.396	0.585	
311	Central African Republic	2016	2.693	6.785	0.290	
10	Afghanistan	2018	2.694	7.692	0.508	
...	...	...	...	...	...	...
450	Denmark	2007	7.834	10.891	0.954	
561	Finland	2018	7.858	10.783	0.962	
563	Finland	2020	7.889	10.750	0.962	
451	Denmark	2008	7.971	10.880	0.954	
449	Denmark	2005	8.019	10.851	0.972	

1925 rows × 11 columns



In [75]: `df.to_csv("cleaned file.csv")`

In []: